

Timing Attacks & Side Channels

The side channel: what your machine leaks without meaning to

A computer leaks information *through physics*: time taken, power used, EM radiated, cache hits/misses. Attackers measuring those physical signals can recover keys – all without 'breaking' the math.

Timing attacks: the accessible one

If 'check password' returns faster when the first char is wrong, an attacker can time how many characters matched:

```
// naive (avoid!)
function check(input) {
  // returns after first mismatch -> timing reveals prefix length
}
// constant-time (use hashes/compare)
crypto.timingSafeEqual(hash(input), hash(stored))
```

Real world:

User enumeration: login for unknown user returns faster/slower

Padding oracle (Bleichenbacher / CBC-padding): server's error *timing* distinguishes 'bad padding' vs 'bad MAC' → decrypt chosen ciphertexts

Padding oracle explained (concept)

```
CBC + server = unpad: if "no, padding bad" response differs
from "yes but MAC bad" → attacker tweaks bytes, oracle leaks:
  manipulate 1 block, observe padding OK/MAC fail,
  infer plaintext byte-by-byte (a byte-at-a-time decryption oracle)
```

Defending against timing

Constant-time code: compare complete buffers (timingSafeEqual), never short-circuit on secret values

AEAD with MAC-then-decrypt (unpad first, MAC after) – the 'different errors' disappear if you return the SAME generic error

Use authenticated modes (GCM) + fixed error text

Normalize login duration (fake work) to kill user-enum timing

Measuring matters

```
python3 - <<'EOF'
# log response times for known vs unknown user (lab)
# typical: p50/p99 difference
EOF
```

TC (timing attack) mitigation checks: run 1000 samples, look for mean shift.

[← Brute Force, Rainbow Tables ...](#)

[Hash Collisions, Length Extens...](#)